Docker, conteneurisation, sécurité

Formation ENSTA 2A — TD2

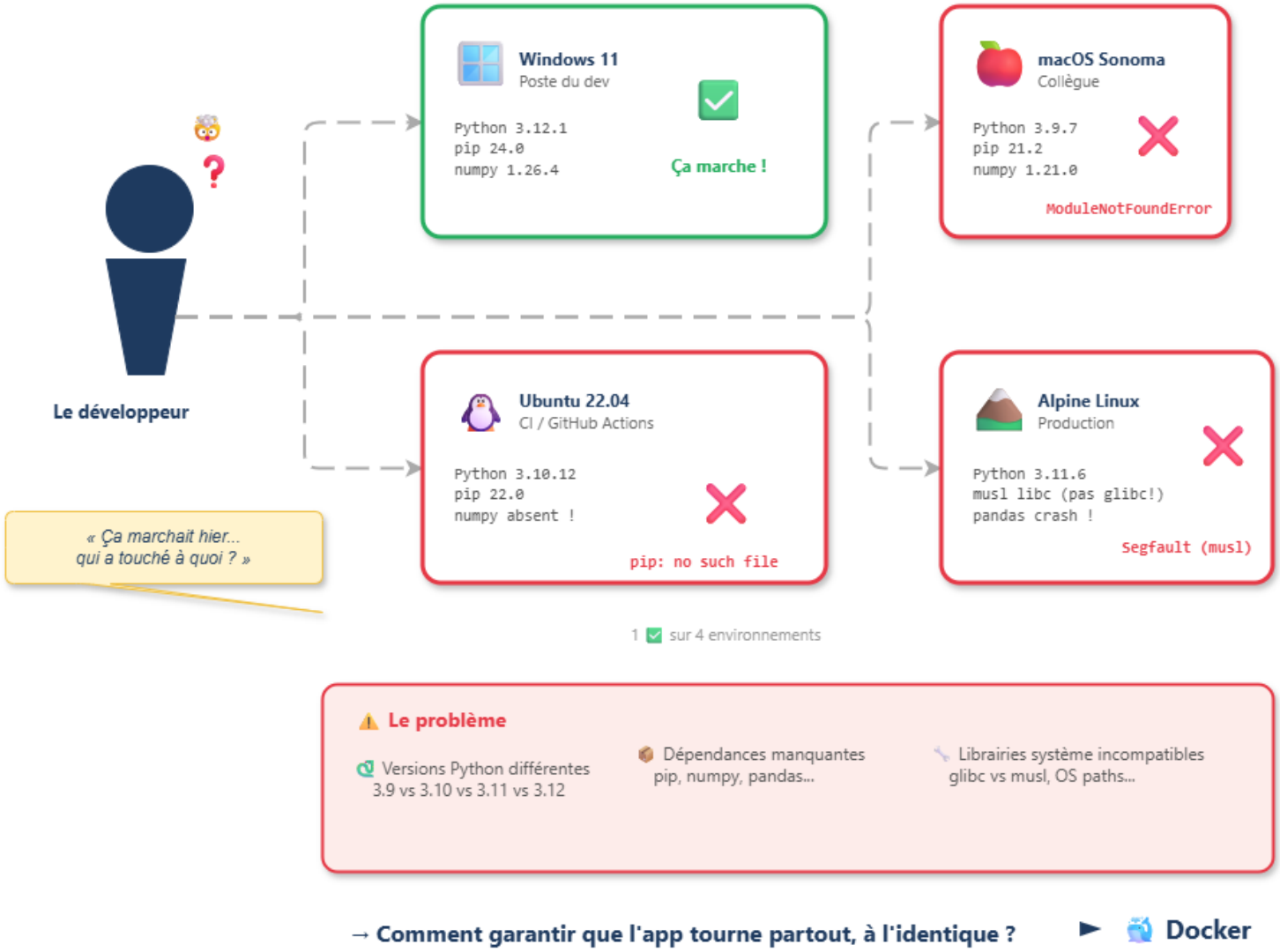
PLATINUM Top 1%
ecovadis
Sustainability Rating
NOV 2024
ISO 27001
BUREAU VERITAS
Certification
VISA

1 Le problème : "Works on my machine“
2 Concepts fondamentaux docker
3 Ecrire un dockerfile
4 Layers et optimisation
5 Commandes essentielles
6 Docker compose
7 Volumes : persistances des données
8 Sous le capot
9 Sécurité docker
10 Conclusion & TD

1. Le problème « Works on my machine »

« Works on my machine »

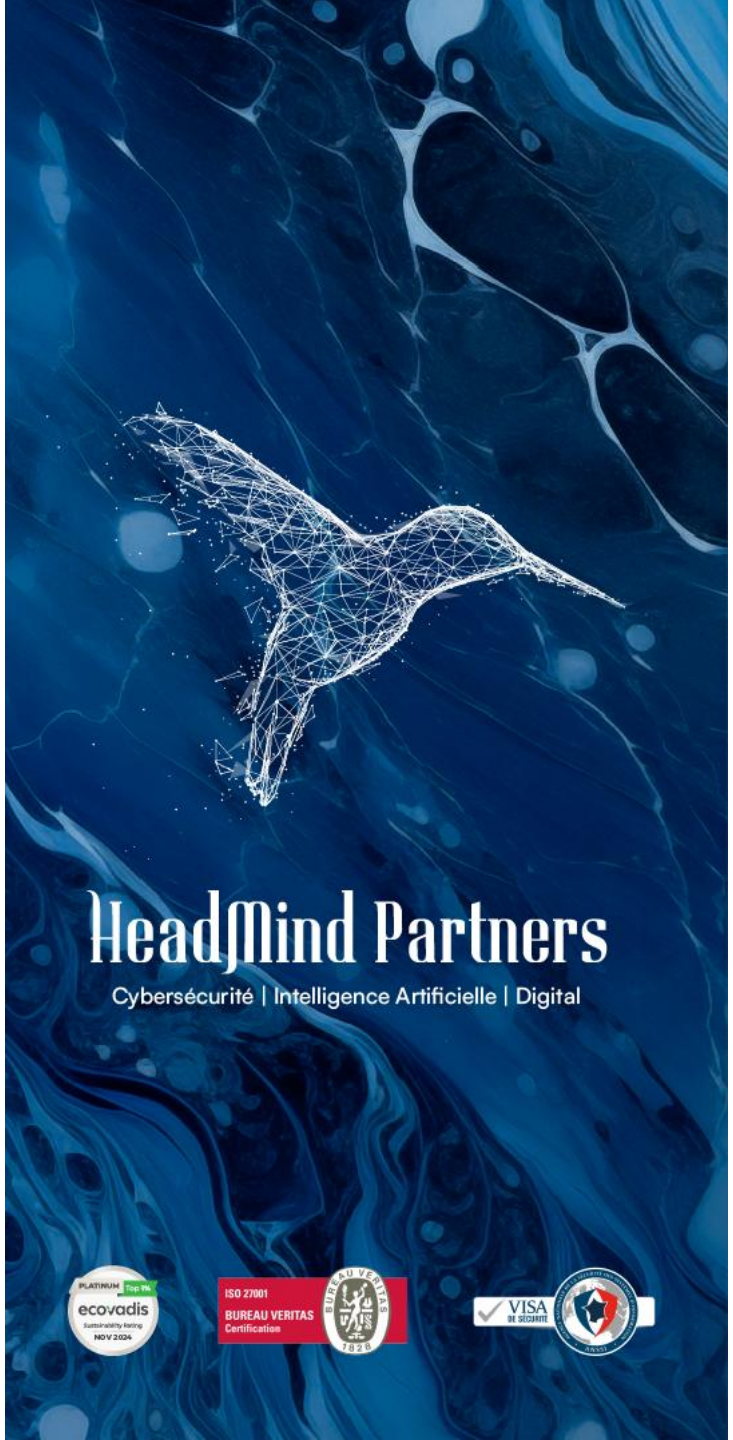
- Dev sur Windows, collègue sur Mac
- CI sur Ubuntu, Prod sur Alpine Linux
- Versions Python différentes
- Dépendances système manquantes
- « Ça marchait hier, qui a touché à quoi ? »



Le problème

- Versions Python différentes 3.9 vs 3.10 vs 3.11 vs 3.12
- Dépendances manquantes pip, numpy, pandas...
- Librairies système incompatibles glibc vs musl, OS paths...

→ Comment garantir que l'app tourne partout, à l'identique ? ► Docker



2. Concepts fondamentaux docker

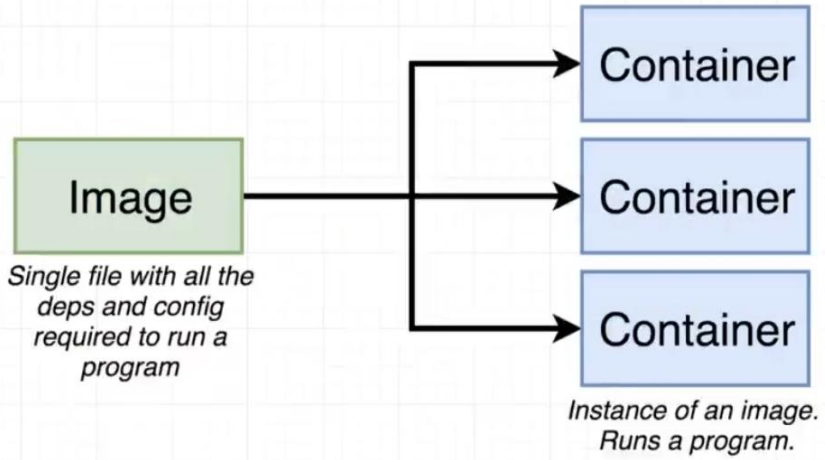
Images VS Containers

Image = Plan de construction de l'app
Ex : python:3.9-slim

- C'est un peu la « description d'une VM »
- Template read-only
- Comme une classe en POO
- Recette de cuisine
- Stockée dans un registry
- Versionnée (tags)
- Décrite dans le « Dockerfile »
  - › python:3.12 -> 900MB
  - › Python:3.12-alpine -> 60MB

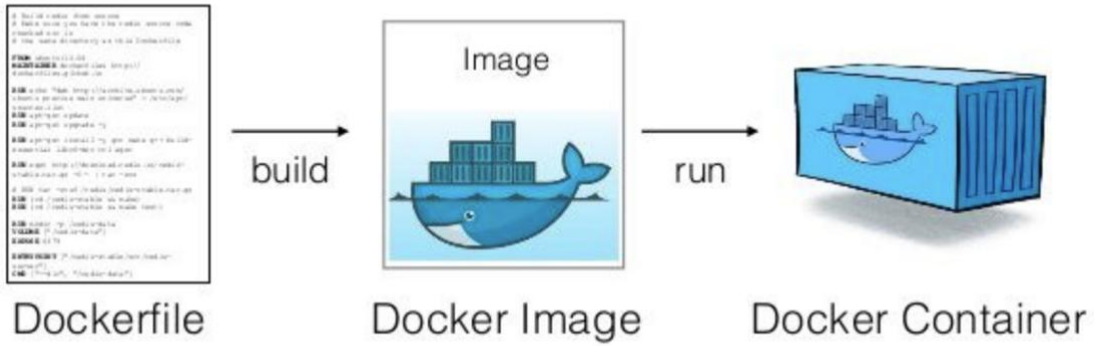
Container = Instance déployée de l'app
Ex : streamlit-thibaud-1d3fdd4

- C'est la "VM" elle même, fait tourner son linux
- Instance en exécution
- Comme un objet en POO
- Éphémère par défaut
- 1 image → N containers
- 1 container python ~ 30MB de RAM



Du dockerfile au container

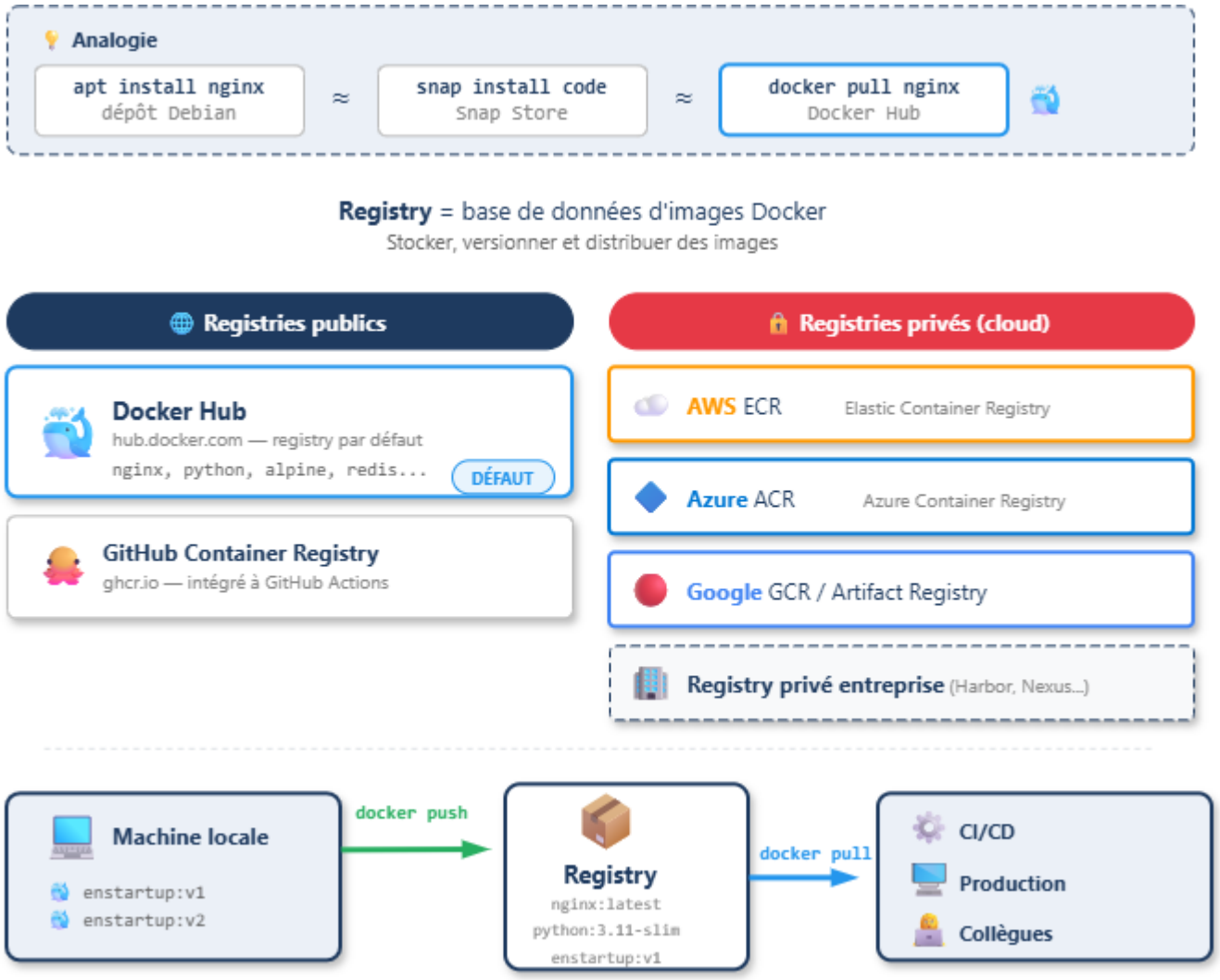
- Dockerfile : recette (instructions texte)
- Image : template construit (layers empilées)
- Container : instance qui tourne
- Tant qu'on a l'image, on peut recréer un container



Autre concept : Le registry pour stocker les images

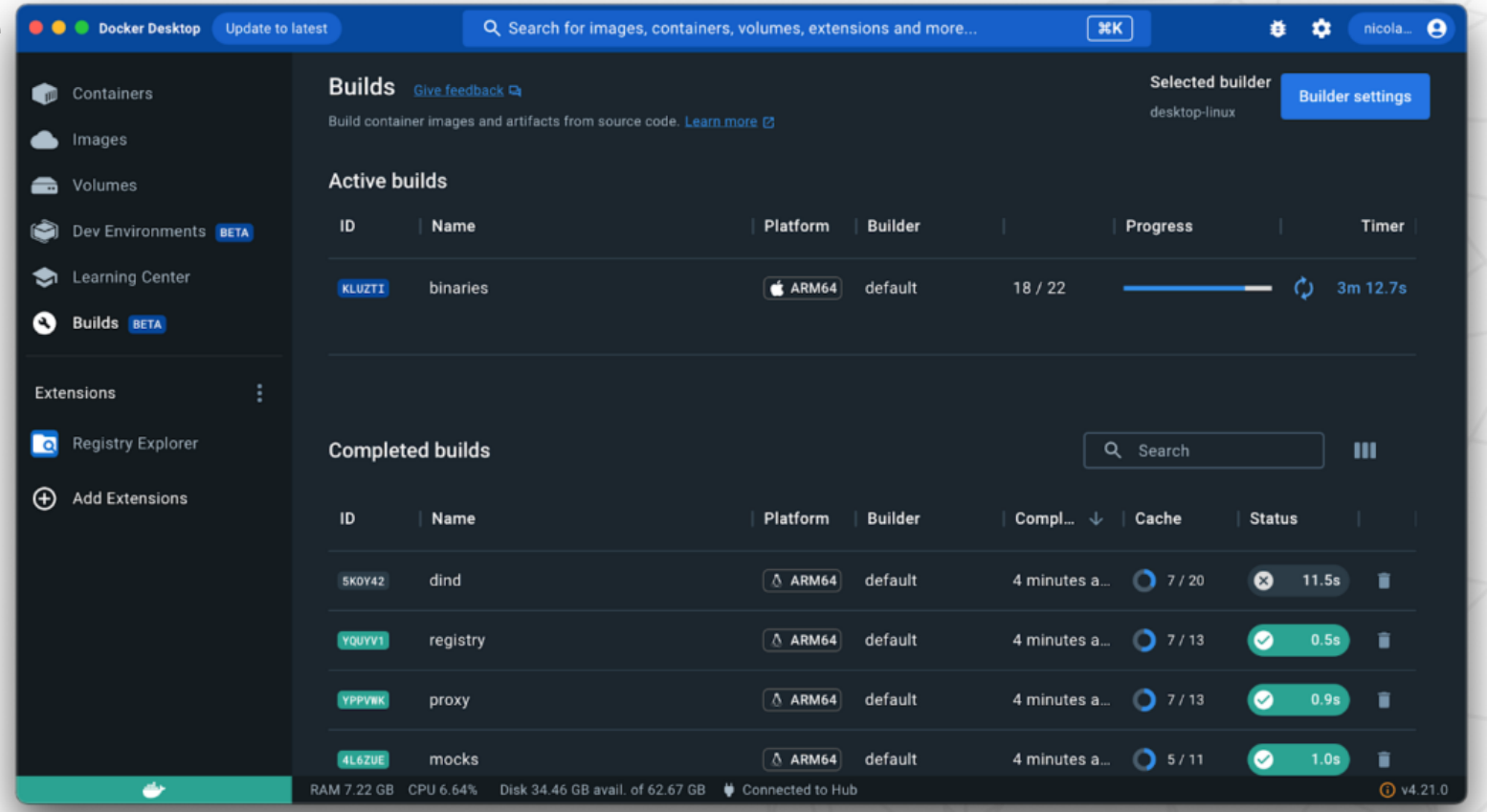
C'est une base publique ou privée d'images

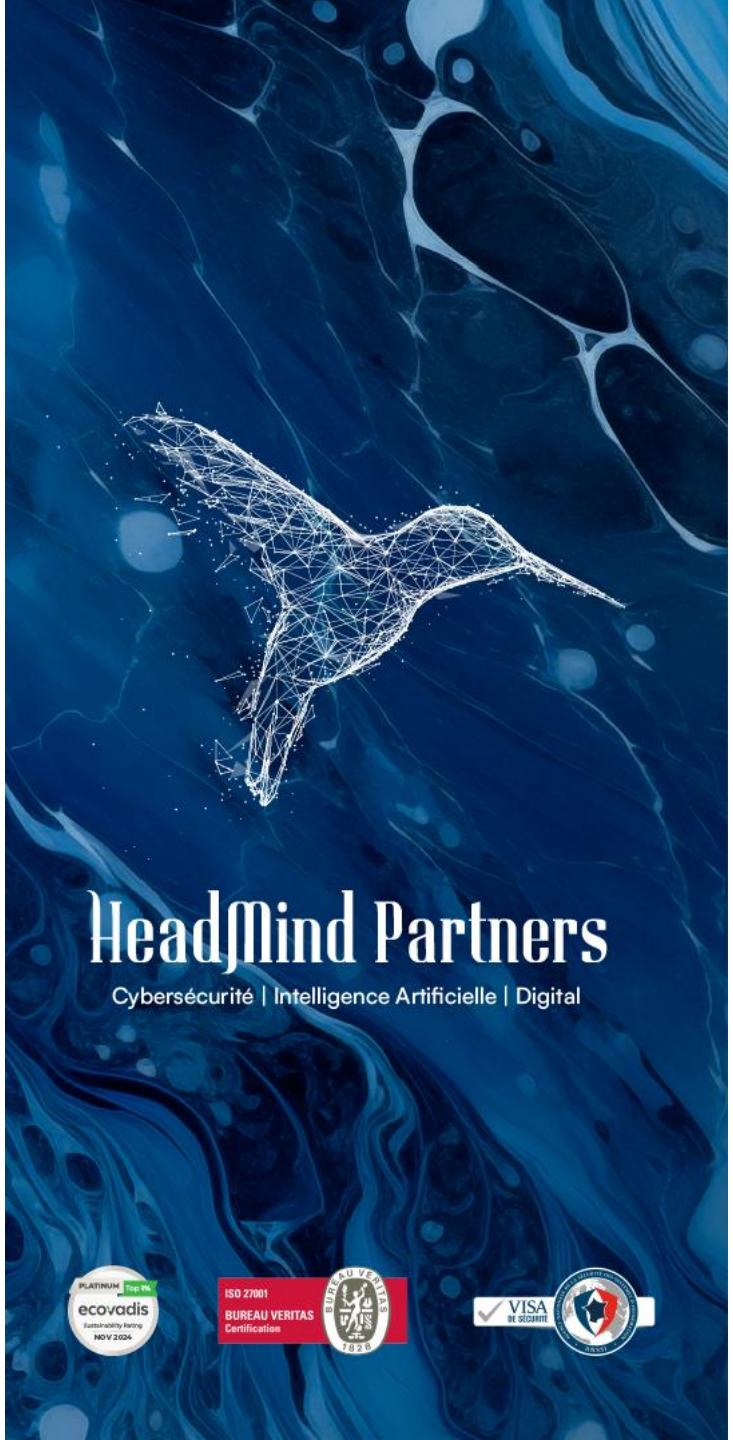
- Docker Hub : registry public par défaut
- GitHub Container Registry (ghcr.io)
- AWS ECR, Azure ACR, Google GCR
- Registry privé (entreprise)
- docker push / docker pull



Installation

- Docker Desktop : Windows / Mac (inclut une VM Linux)
- Docker Engine : Linux natif
- Extension VS Code recommandée
- Vérifier : docker --version
- Test : docker run hello-world





3. Ecrire un Dockerfile

Structure d’un Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

ENV PORT=8501
EXPOSE 8501

CMD ["streamlit", "run", "app.py"]
```

Chaque instruction crée une couche (layer) dans l'image

Instructions clés

Instruction	Rôle	Exemple
FROM	Image de base	FROM python:3.11-slim
WORKDIR	Répertoire de travail	WORKDIR /app
COPY	Copier des fichiers	COPY . .
RUN	**Exécuter (build time)**	**RUN pip install -r requirements.txt**
ENV	Variable d'environnement	ENV PORT=8501
EXPOSE	Documenter le port	EXPOSE 8501
CMD	**Commande par défaut (run time)**	**CMD ["python", "app.py"]**

RUN vs CMD vs ENTRYPOINT

temps →

🔨 BUILD TIME — docker build

RUN

- ✦ S'exécute pendant le **build**
- ✦ Crée une nouvelle **layer** à chaque fois
- ✦ Peut apparaître **plusieurs fois**

```
RUN useradd --create-home appuser      + layer
COPY . .
RUN pip install -r requirements.txt    + layer
FROM python:3.11-slim
```

figé dans l'image

▶ RUN TIME — docker run

ENTRYPOINT

- ✦ S'exécute au **lancement** du container
- ✦ Définit le **processus principal**
- ✦ **Non overridable** (sauf --entrypoint)
- ✦ La dernière instruction compte

🔒 `ENTRYPOINT ["streamlit", "run"]` FIXE

pas de layer

CMD

- ✦ S'exécute au **lancement** du container
- ✦ Commande / arguments **par défaut**
- ✦ **Overridable** au docker run
- ✦ La dernière instruction compte

🔓 `CMD ["app.py"]` DÉFAUT

pas de layer

🧩 Combinaison

Dockerfile :

```
ENTRYPOINT ["streamlit", "run"]
+
CMD ["app.py"]
```

↓

Par défaut

```
$ docker run monimage
→ streamlit run app.py
```

Override CMD

```
$ docker run monimage autre.py
→ streamlit run autre.py
```

■ ENTRYPOINT (fixe) ■ CMD (remplaçable)

● **RUN** = installer, configurer, préparer
Le chef qui prépare la cuisine avant le service

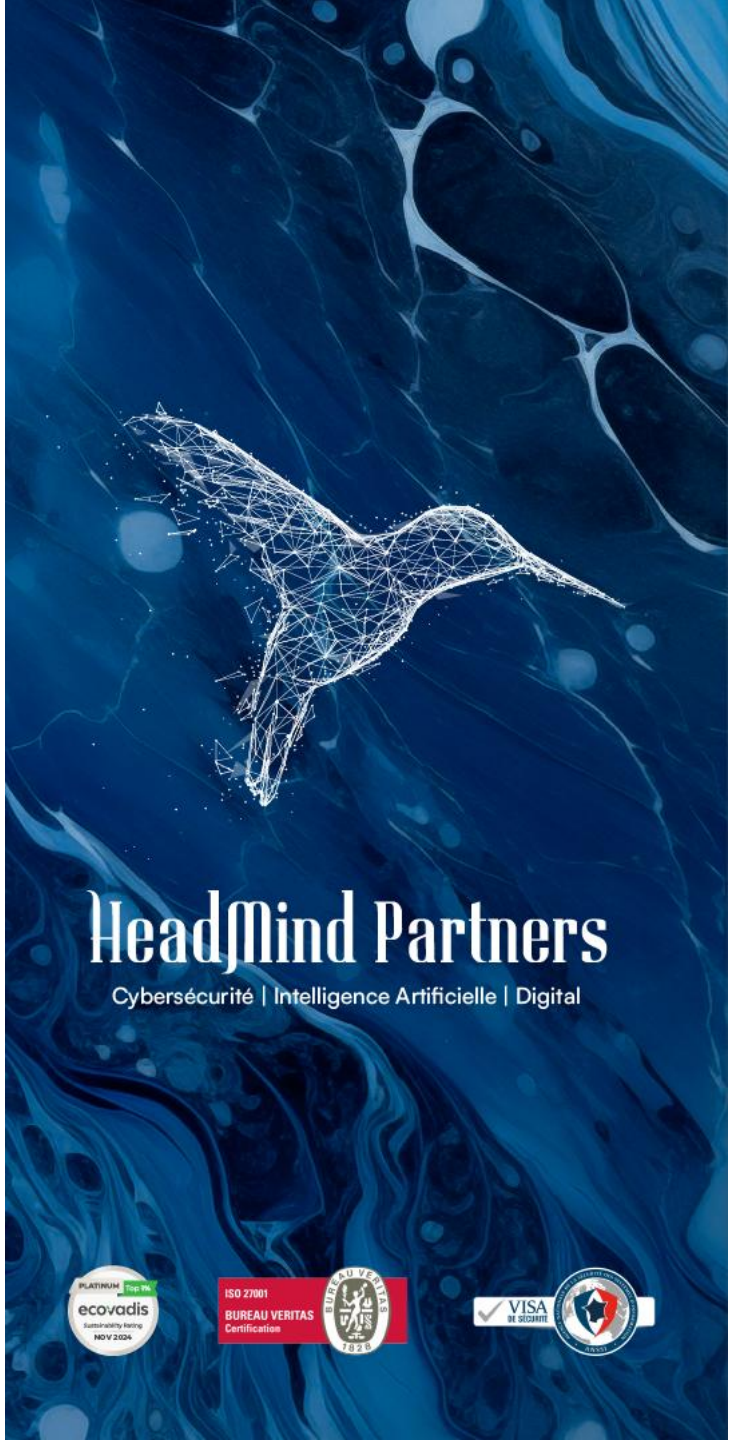
● **ENTRYPOINT** = le programme à lancer
Le serveur qui prend les commandes (toujours le même)

● **CMD** = les arguments par défaut
Le plat du jour (le client peut changer)

.dockerignore

```
# Fichiers Python
__pycache__/
*.pyc
*.pyo
venv/
.env

# Git
.git/
.gitignore

# IDE
.vscode/
.idea/
```

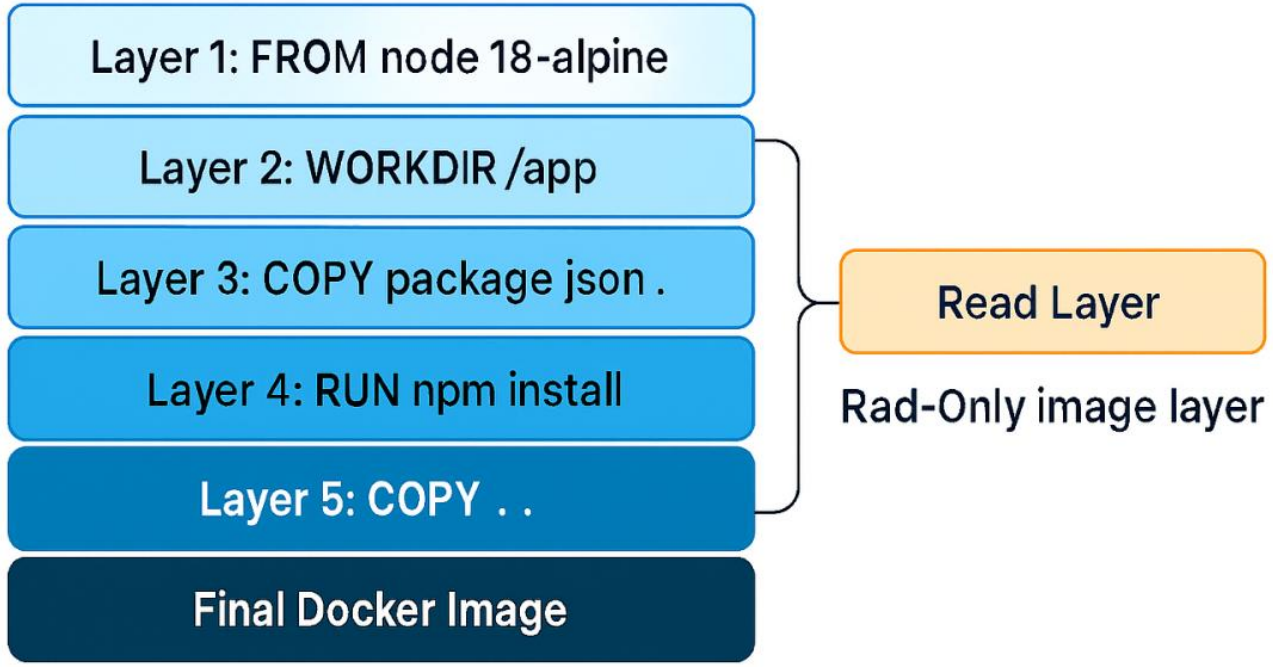
Comme .gitignore : exclure les fichiers inutiles de l'image
On peut faire COPY . . Sans risque ensuite



4. Layers et optimisations

Concept : Les layers

- Chaque instruction Dockerfile = une couche (layer)
- Les layers sont mis en cache et réutilisés
- Si un layer change → tous les suivants sont reconstruits
- docker history <image> : voir les layers



✕ Mauvais ordre (rebuild tout à chaque changement)

```
FROM python:3.11-slim
WORKDIR /app

# Si app.py change, on réinstalle TOUT
COPY . .
RUN pip install -r requirements.txt

CMD ["python", "app.py"]
```

Problème : le moindre changement de code invalide le cache des dépendances

✓ Bon ordre (cache optimisé)

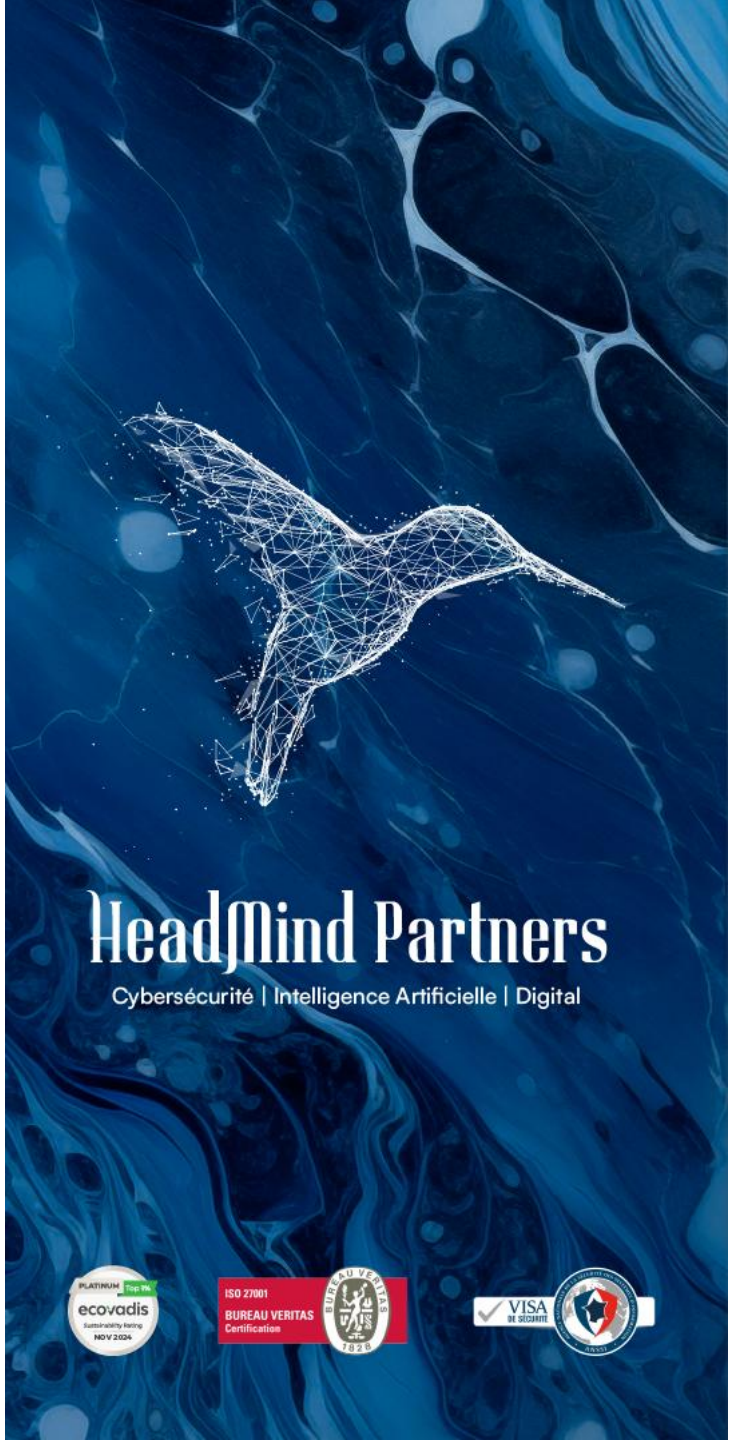
```
FROM python:3.11-slim
WORKDIR /app

# D'abord les dépendances (changent rarement)
COPY requirements.txt .
RUN pip install -r requirements.txt

# Ensuite le code (change souvent)
COPY . .

CMD ["python", "app.py"]
```

Les dépendances sont mises en cache, seul le code est recopié

Visualiser les layers

```
$ docker history mon-app:v1

IMAGE          CREATED        SIZE      COMMAND
a1b2c3d4e5f6   2 hours ago    5.2MB     CMD ["python" "app.py"]
b2c3d4e5f6g7   2 hours ago    0B        EXPOSE 8501
c3d4e5f6g7h8   2 hours ago    45MB      RUN pip install -r requirements.txt
d4e5f6g7h8i9   2 hours ago    1.2KB     COPY requirements.txt .
e5f6g7h8i9j0   2 hours ago    0B        WORKDIR /app
f6g7h8i9j0k1   3 days ago     125MB     python:3.11-slim
```

Chaque ligne = une layer avec sa taille



5. Commandes docker essentielles

Commandes essentielles

```
# Construire une image
$ docker build -t mon-app:v1 .

# Tagger pour un registry
$ docker tag mon-app:v1 ghcr.io/username/mon-app:v1

# Pousser sur le registry
$ docker push ghcr.io/username/mon-app:v1

# Récupérer une image
$ docker pull python:3.11-slim
```

Docker run : lancer un conteneur

```
# Basique
$ docker run mon-app:v1

# Exposer un port (-p host:container)
$ docker run -p 8501:8501 mon-app:v1

# Mode détaché (background)
$ docker run -d --name app mon-app:v1

# Avec variables d'environnement
$ docker run -e DATABASE_URL=postgres://... mon-app:v1

# Mode interactif (shell)
$ docker run -it python:3.11 bash
```

-p = port, -d = detached, -e = env, -it = interactive terminal

Inspecter, debug

```
# Voir les containers actifs
$ docker ps

# Voir tous les containers (même arrêtés)
$ docker ps -a

# Logs d'un container
$ docker logs mon-container
$ docker logs -f mon-container  # follow (temps réel)

# Entrer dans un container
$ docker exec -it mon-container bash

# Détails complets (JSON)
$ docker inspect mon-container
```

Nettoyage

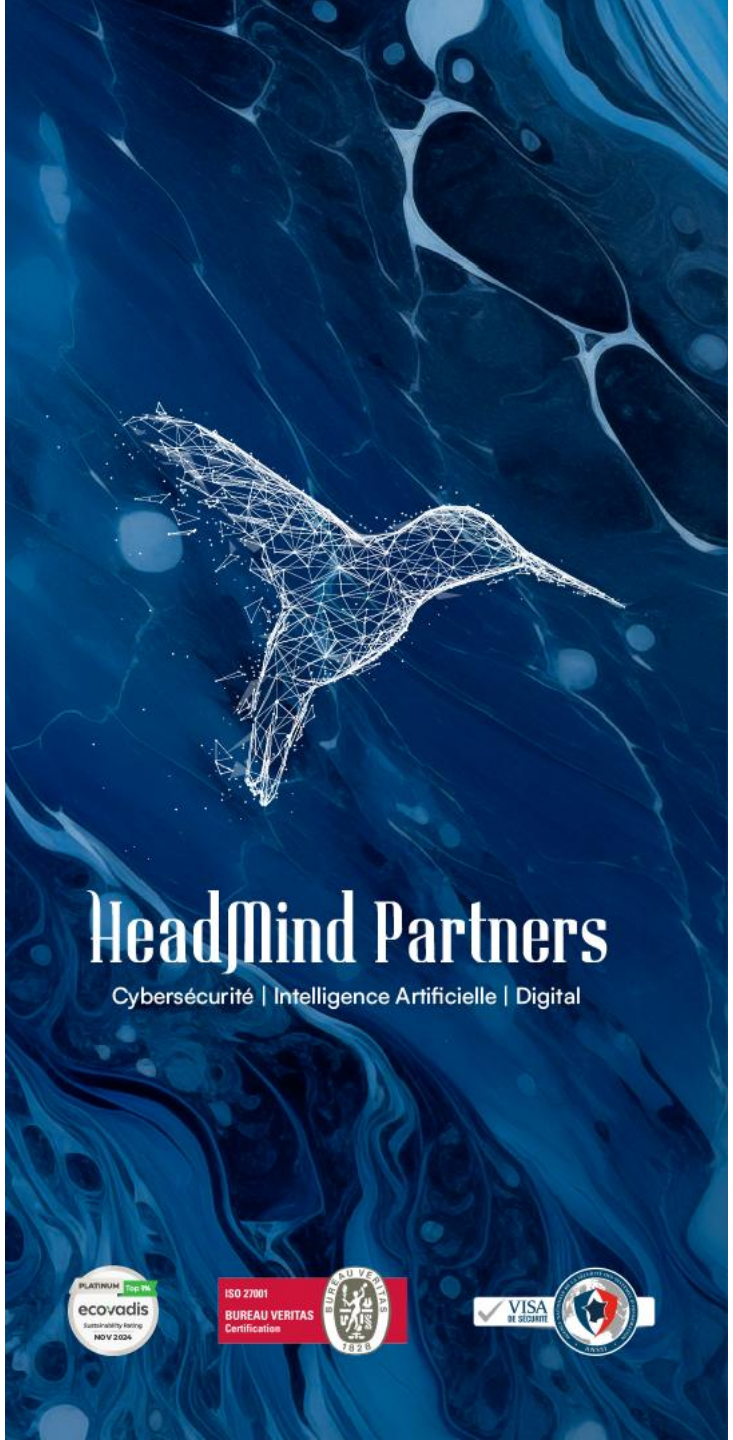
Attention, point sujet à confusion
Quand on stoppe un conteneur, ses fichiers persistent, pour facilliter un redémarrage
Il faut donc le supprimer
On peut supprimer un conteneur OU une image

```
# Arrêter un container
$ docker stop mon-container

# Supprimer un container
$ docker rm mon-container

# Supprimer une image
$ docker rmi mon-app:v1

# Tout nettoyer (containers arrêtés, images orphelines, cache)
$ docker system prune

# Nettoyer TOUT (attention!)
$ docker system prune -a
```

Attention : prune -a supprime aussi les images non utilisées

DevSecOps : Docker scout

```
scout-demo-service-main — jonathanroberts@DockerJbertsMBP — ..-service-main — -zsh — 94×19
→  scout-demo-service-main docker scout quickview  docker/scout-demo-service:jr

    ✓ SBOM of image already cached, 107 packages indexed

  Your image  docker/scout-demo-service:jr   |   2C   17H   6M   0L   2?
  Base image  alpine:3                       |   2C   15H   6M   0L   2?
  Refreshed base image  alpine:3             |   0C    0H   0M   0L   1?
                                             |   -2   -15   -6        -1
  Updated base image  alpine:3.16            |   0C    0H   0M   0L   1?
                                             |   -2   -15   -6        -1

    Know more about vulnerabilities:
      docker scout cves docker/scout-demo-service:jr
    Know more about base image update recommendations:
      docker scout recommendations docker/scout-demo-service:jr
→  scout-demo-service-main
```



6. Docker compose

Pourquoi « docker compose » ?

- Une app = souvent plusieurs services (donc plusieurs conteneurs)
  - Exemple : nginx (frontend) <->fastAPI (backend) <-> PostgreSQL (BDD)
- App + Base de données + Cache + …
- Difficile de gérer manuellement plusieurs docker run
- Compose : orchestrer plusieurs containers
- Un fichier YAML pour tout décrire

Sans Compose

1
```
docker network create myapp-net
```

2
```
docker run -d \
  --name postgres \
  --network myapp-net \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql \
  postgres:16
```

3
```
docker run -d \
  --name redis \
  --network myapp-net \
  redis:alpine
```

4
```
docker build -t myapp:v1 .
```

5
```
docker run -d \
  --name app \
  --network myapp-net \
  -p 8501:8501 \
  -e REDIS_HOST=redis \
  myapp:v1
```

6
```
docker run -d \
  --name nginx \
  --network myapp-net \
  -p 80:80 nginx:alpine
```

- ❌ Ordre de lancement à respecter
- ❌ Réseau à créer manuellement
- ❌ Volumes à gérer séparément
- ❌ 6 commandes à retenir
- ❌ Impossible à reproduire facilement

Avec Compose

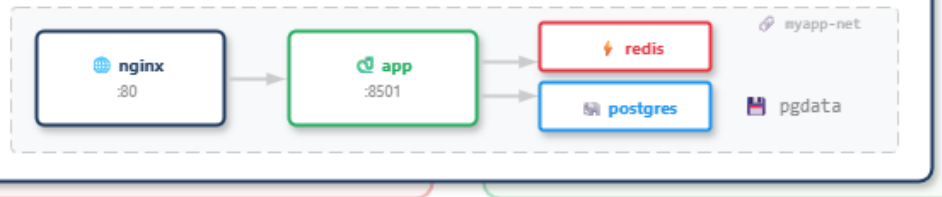
docker-compose.yml

```yaml
services:

  app:
    build: .
    ports:
      - "8501:8501"
    depends_on:
      - redis
      - postgres
    environment:
      - REDIS_HOST=redis

  redis:
    image: redis:alpine

  postgres:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD=secret
    volumes:
      - pgdata:/var/lib/postgresql

volumes:
  pgdata:
```

```
$ docker compose up
```

- ✅ Réseau créé automatiquement
- ✅ Ordre de démarrage respecté
- ✅ Volumes gérés
- ✅ Un fichier = toute l'infra
- ✅ Reproductible par toute l'équipe

Résultat : 3 containers + 1 réseau + 1 volume

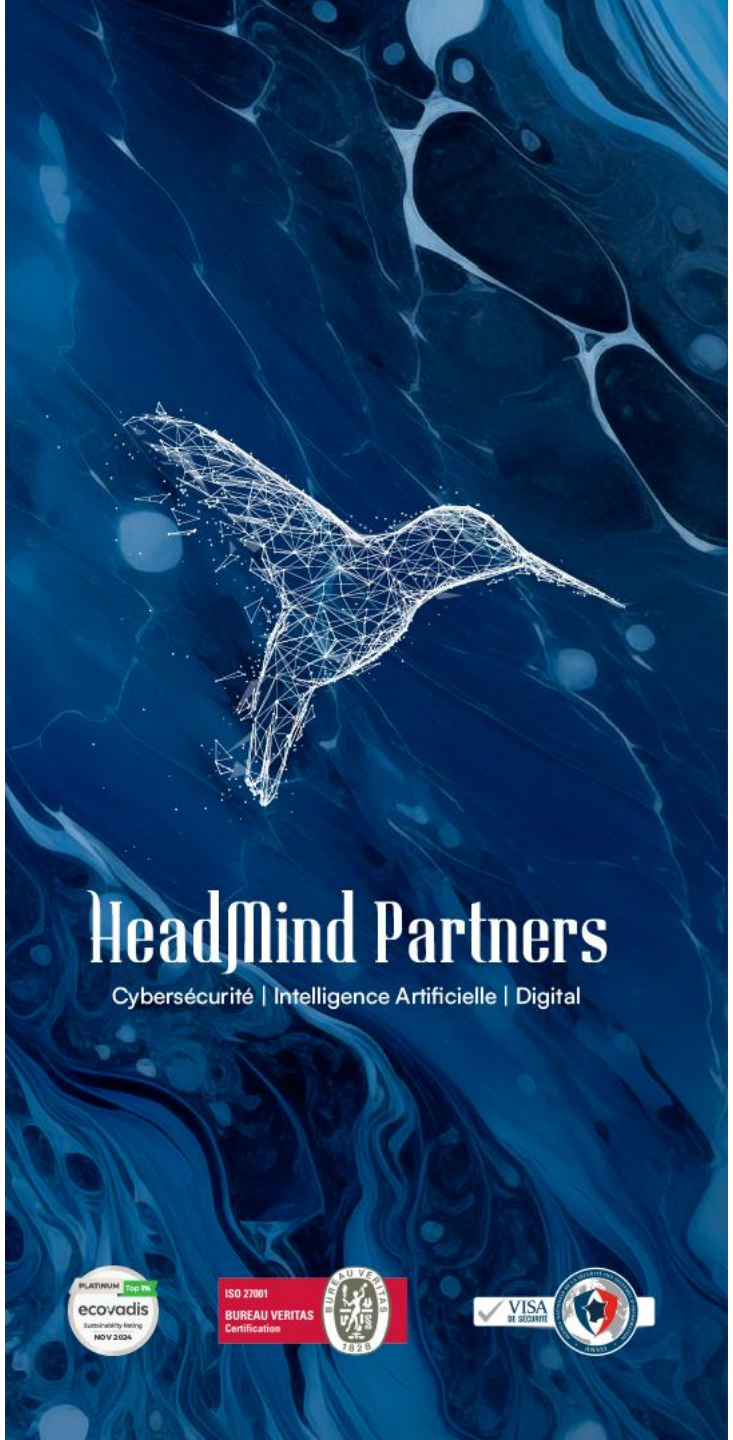


docker-compose.yml

```yaml
services:
  app:
    build: .
    ports:
      - "8501:8501"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/enstartup
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: enstartup
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Commandes Compose

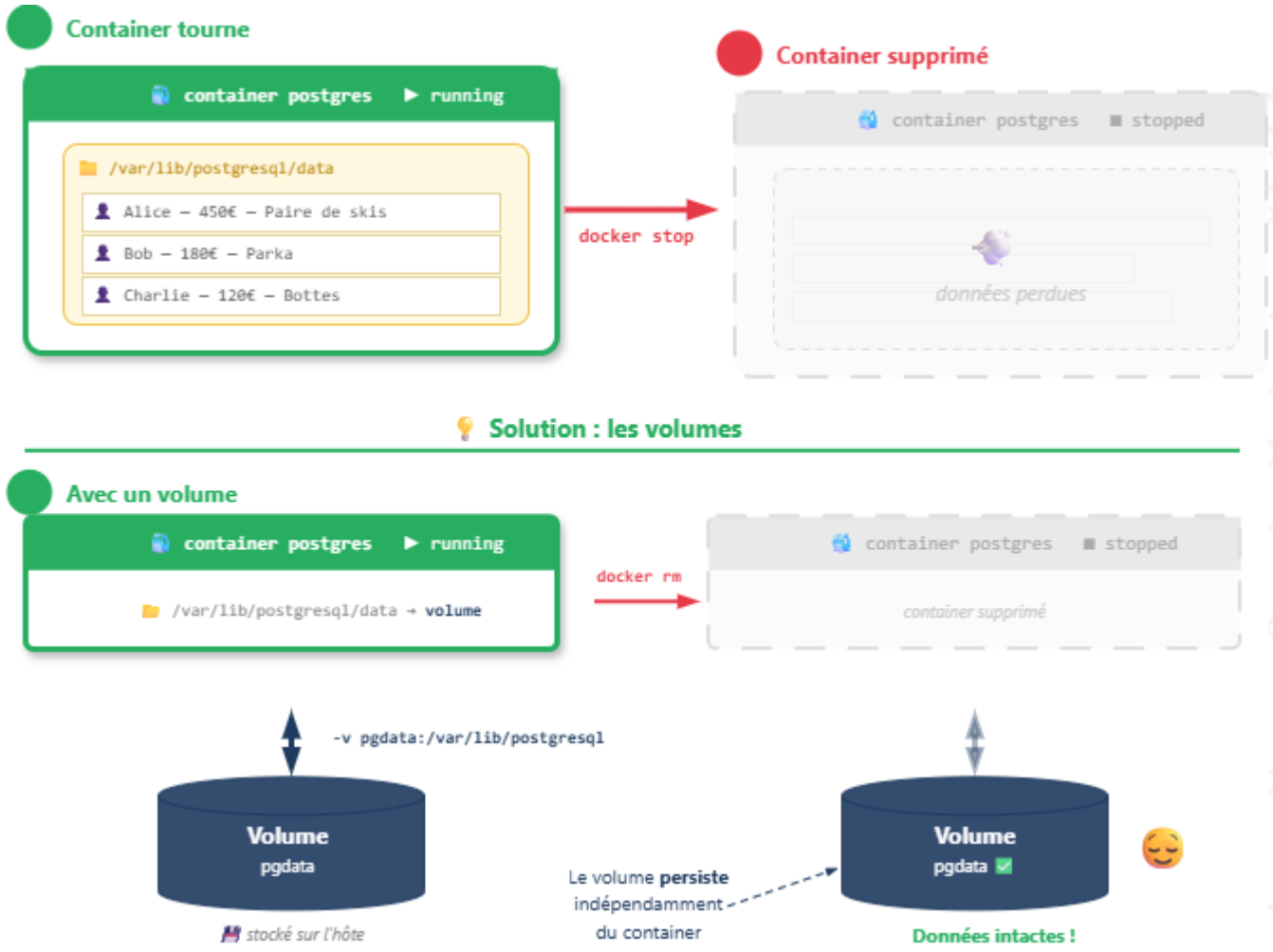
```
# Démarrer tous les services
$ docker compose up

# En arrière-plan
$ docker compose up -d

# Voir les logs
$ docker compose logs
$ docker compose logs -f app  # suivre un service

# Arrêter
$ docker compose down

# Arrêter et supprimer les volumes
$ docker compose down -v
```


HeadMind Partners
Cybersécurité | Intelligence Artificielle | Digital
ecovadis
ISO 27001
BUREAU VERITAS
Certification
VISA

7. Volumes

Problème : Données éphémères

- Un container est éphémère par défaut
- Quand il s'arrête, les données sont perdues
- Base de données : on veut garder les données !
- Fichiers uploadés : idem
- Solution : les volumes



Types de volumes

Bind mount

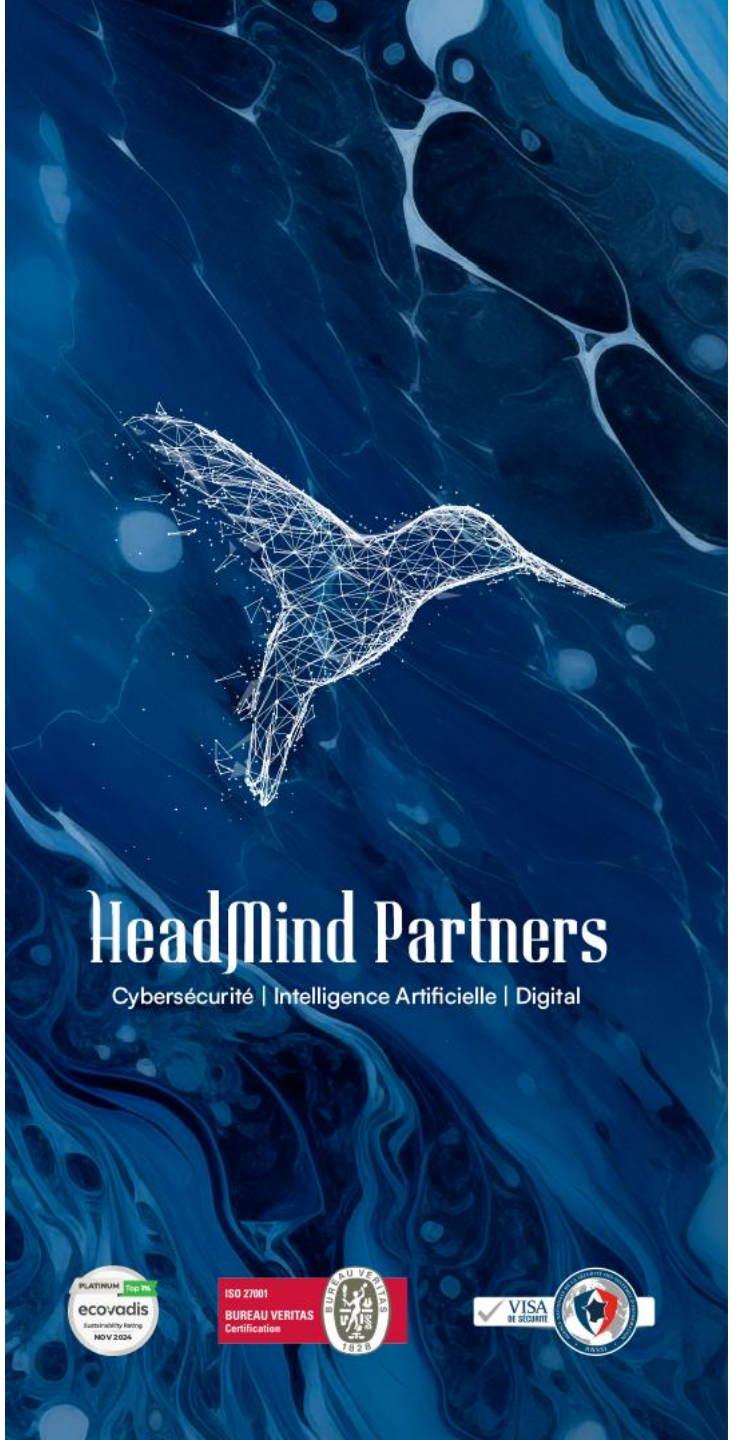
- Monte un dossier de l'hôte
- -v ./data:/app/data
- Idéal pour le dev
- Modifications visibles en temps réel
- Dépend du chemin sur l'hôte

Named volume

- Géré par Docker
- -v db_data:/var/lib/postgresql/data
- Idéal pour la prod
- Portable entre machines
- docker volume ls pour lister

Exemple : persister PostgreSQL

```
# Avec docker run
$ docker run -v db_data:/var/lib/postgresql/data postgres:15

# Avec Compose
services:
  db:
    image: postgres:15
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:  # Déclaration du volume

# Lister les volumes
$ docker volume ls

# Inspecter un volume
$ docker volume inspect db_data
```



8. Sous le capot

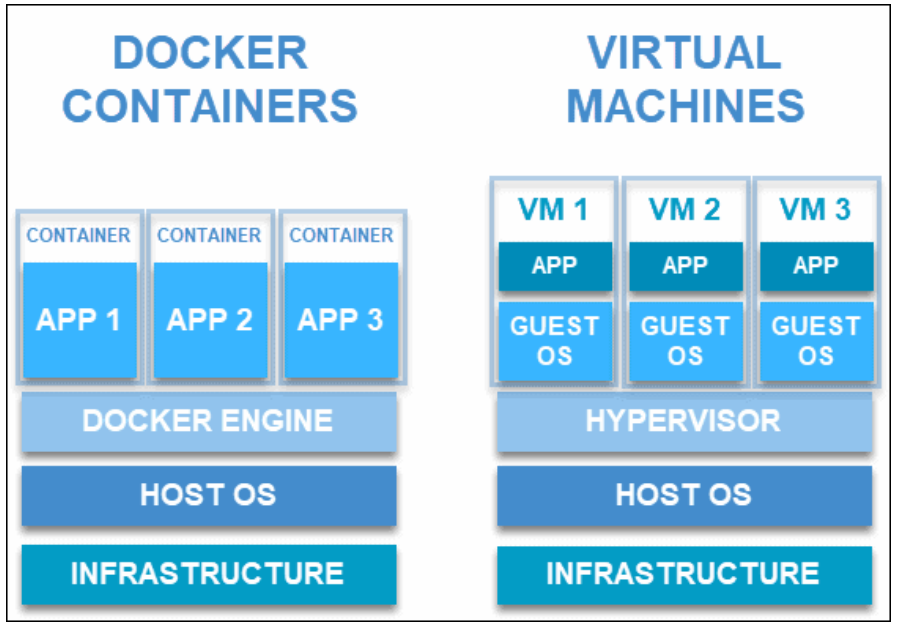
Machine virtuelle V.S. Container

Machine Virtuelle

- OS complet (Go de RAM)
- Hyperviseur (VMware, VirtualBox)
- Démarrage en minutes
- Isolation forte (kernel séparé)
- Overhead important

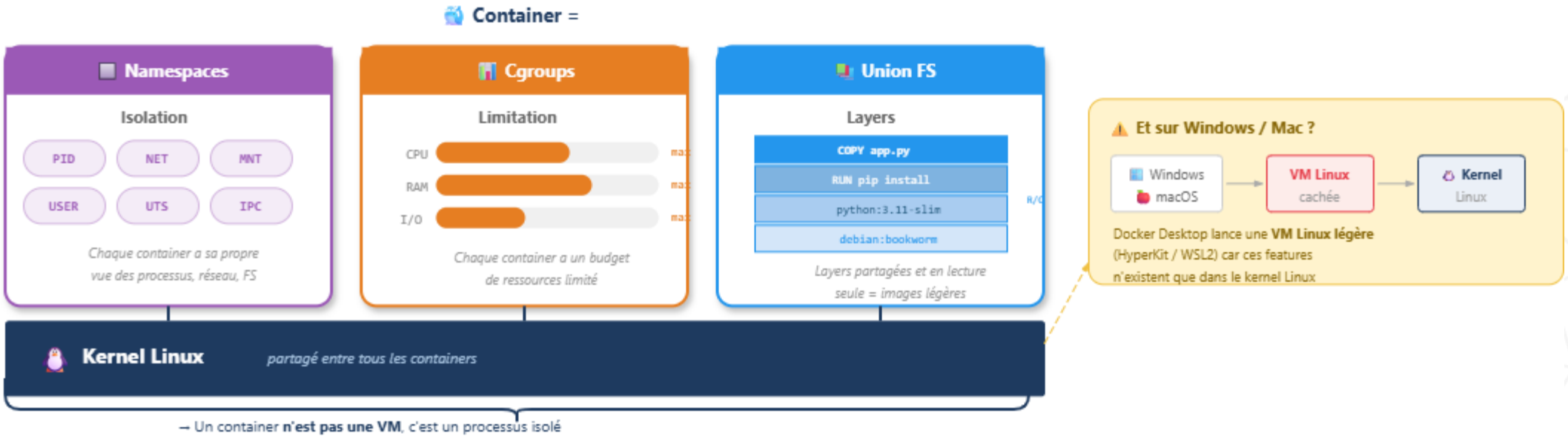
Container

- Partage le kernel de l'hôte
- Container runtime (Docker)
- Démarrage en secondes
- Isolation légère (namespaces)
- Très efficace en ressources



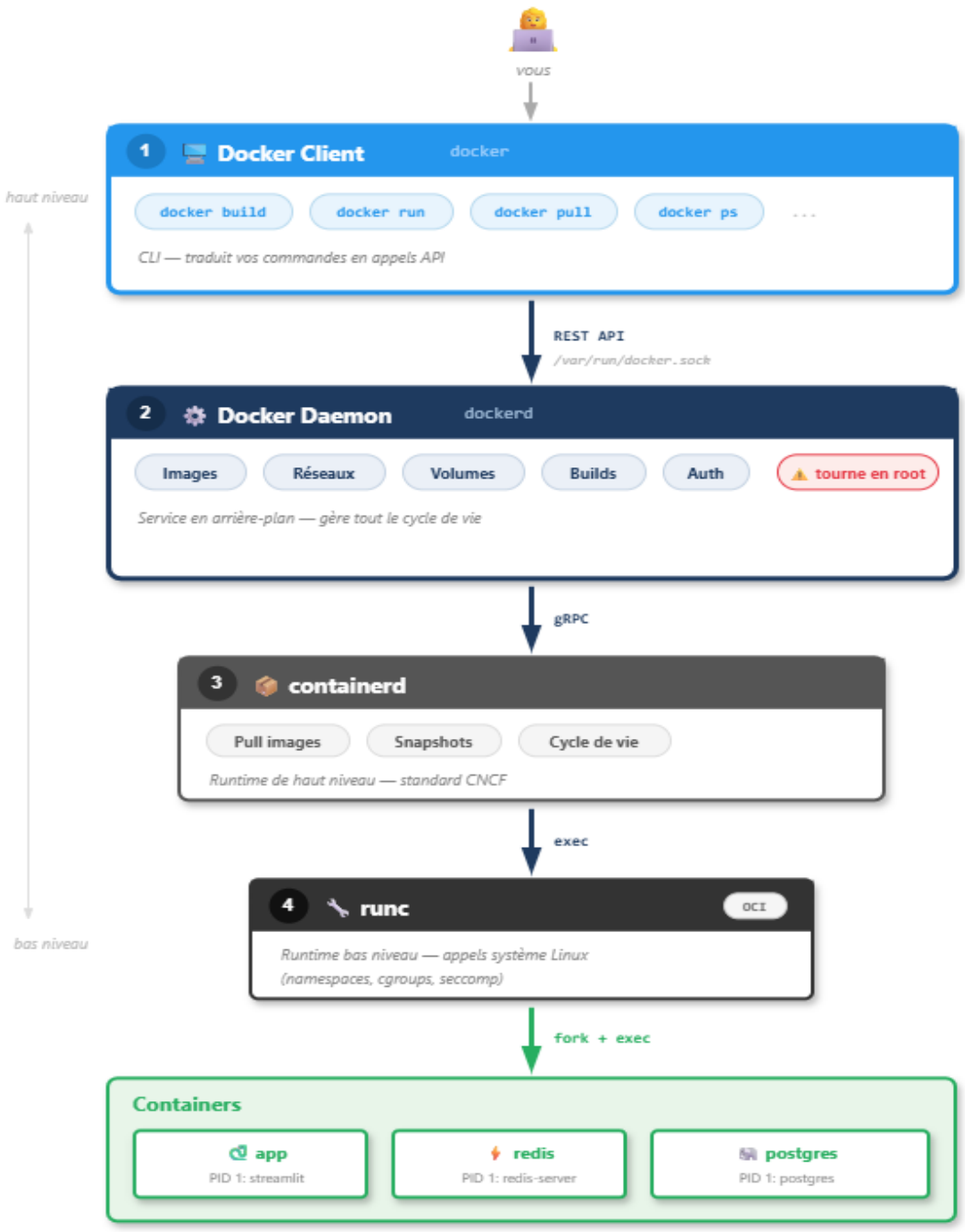
Concepts Linux sous-jacents

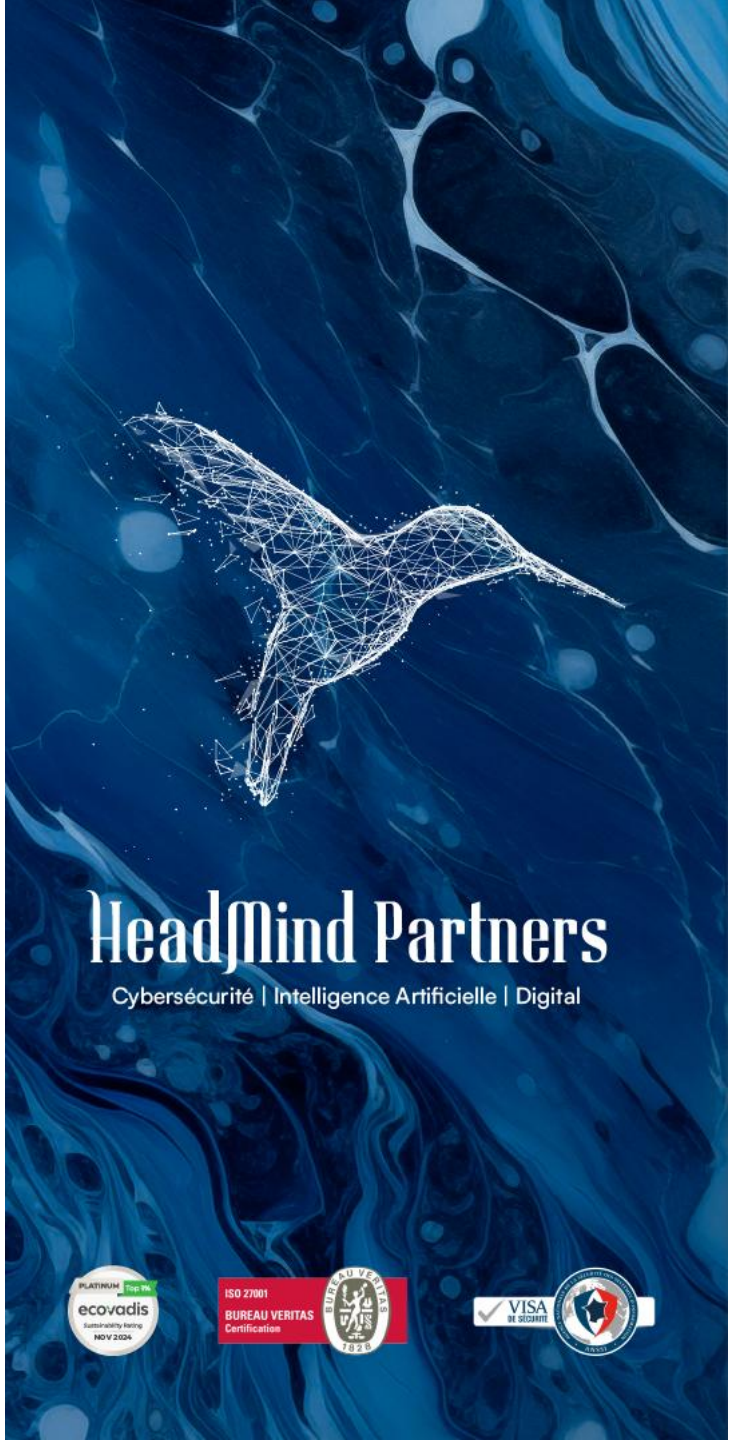
- Namespaces : isolation des processus (PID, network, filesystem)
- Cgroups : limitation des ressources (CPU, RAM, I/O)
- Union filesystem : layers superposées (overlay2)
- → Docker ne fonctionne nativement que sur Linux
- → Sur Windows/Mac, Docker Desktop lance une VM Linux cachée



Architecture Docker

- Docker Client : commandes (docker build, run, ...)
- Docker Daemon : service qui gère les containers
- containerd : runtime bas niveau
- runc : crée les containers (appels système Linux)
- Communication via socket Unix ou TCP





9. La sécurité dans Docker

Le groupe docker = root déguisé

- sudo usermod -aG docker $USER

→ Donne un accès ROOT sur la machine hôte !

- Pourquoi ? Le daemon Docker tourne en root
- Un container peut monter n'importe quel dossier de l'hôte
- Démo : accéder à /etc/shadow depuis un container

```
$ sudo usermod -aG docker $USER
```

« Juste pour utiliser Docker sans sudo, rien de grave... »

en réalité...

Chaîne d'exploitation

1. L'utilisateur est dans le groupe **docker**

peut parler au daemon

2. Monte **tout le filesystem** de l'hôte dans un container

```
docker run -v /:/host alpine
```

le daemon monte en root

3. **Accès ROOT total** sur la machine hôte

Démo : lire /etc/shadow

```
etudiant@server:~
etudiant$ whoami
etudiant
etudiant$ cat /etc/shadow
Permission denied          ✗ accès refusé
etudiant$ docker run -v /:/h alpine
/ # cat /h/etc/shadow
root:$6$xyz...hashed_pwd:19741
etudiant:$6$abc...hashed:19742
✓ hash des mots de passe !
```

groupe docker = accès root sans sudo

Pourquoi ?

Le daemon **dockerd** tourne en **root**

Groupe docker = droit de parler au daemon via le socket

Le daemon exécute vos demandes avec ses privilèges **root**

Solutions

- **Podman** — rootless par défaut
- **Rootless** — Docker sans daemon root

→ On testera Podman dans le TD !

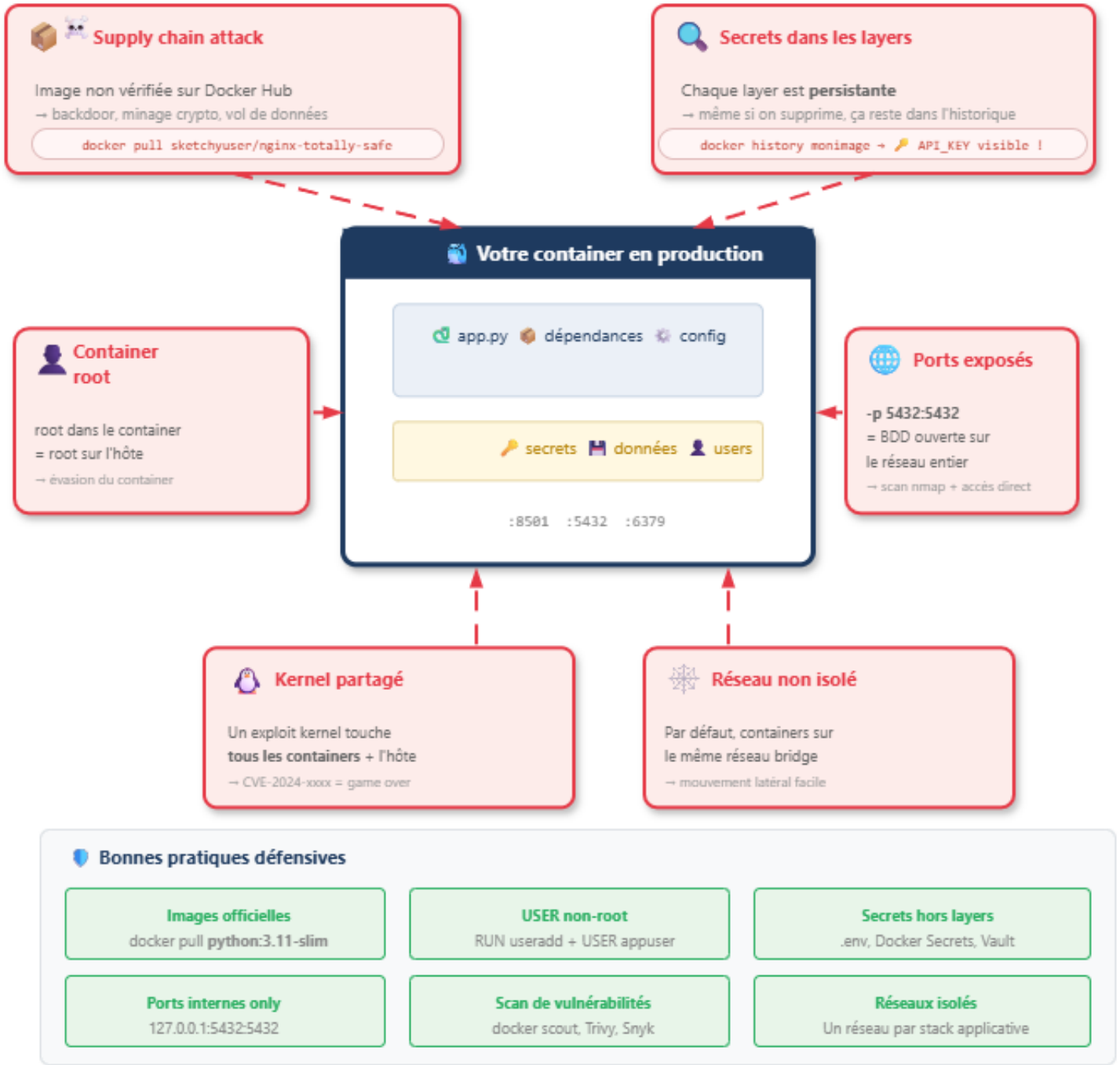
Exploit : lire /etc/shadow

```
# N'importe quel membre du groupe docker peut faire :
$ docker run -v /:/host alpine cat /host/etc/shadow

root:$6$xyz....:19000:0:99999:7:::
daemon:*:19000:0:99999:7:::
thibaud:$6$abc....:19500:0:99999:7:::

# On a accès à TOUS les fichiers de l'hôte !
# Y compris les mots de passe hashés

# On peut même devenir root :
$ docker run -v /:/host -it alpine chroot /host
```

Le groupe docker = accès root sans sudo

Autres risques de sécurité

- Images non vérifiées : supply chain attack
- Secrets dans les layers : docker history les expose
- Container root = processus root sur l'hôte
- Kernel partagé = surface d'attaque
- Ports exposés = accessibles depuis le réseau



Bonnes pratiques sécurité

Risque	Mitigation
Images malveillantes	Images officielles, docker scout
Secrets dans l'image	Variables d'env, Docker secrets, .dockerignore
Container root	Instruction USER dans Dockerfile
Groupe docker dangereux	Podman rootless
Vulnérabilités	Mettre à jour les images de base

Bonne pratique : USER non-root

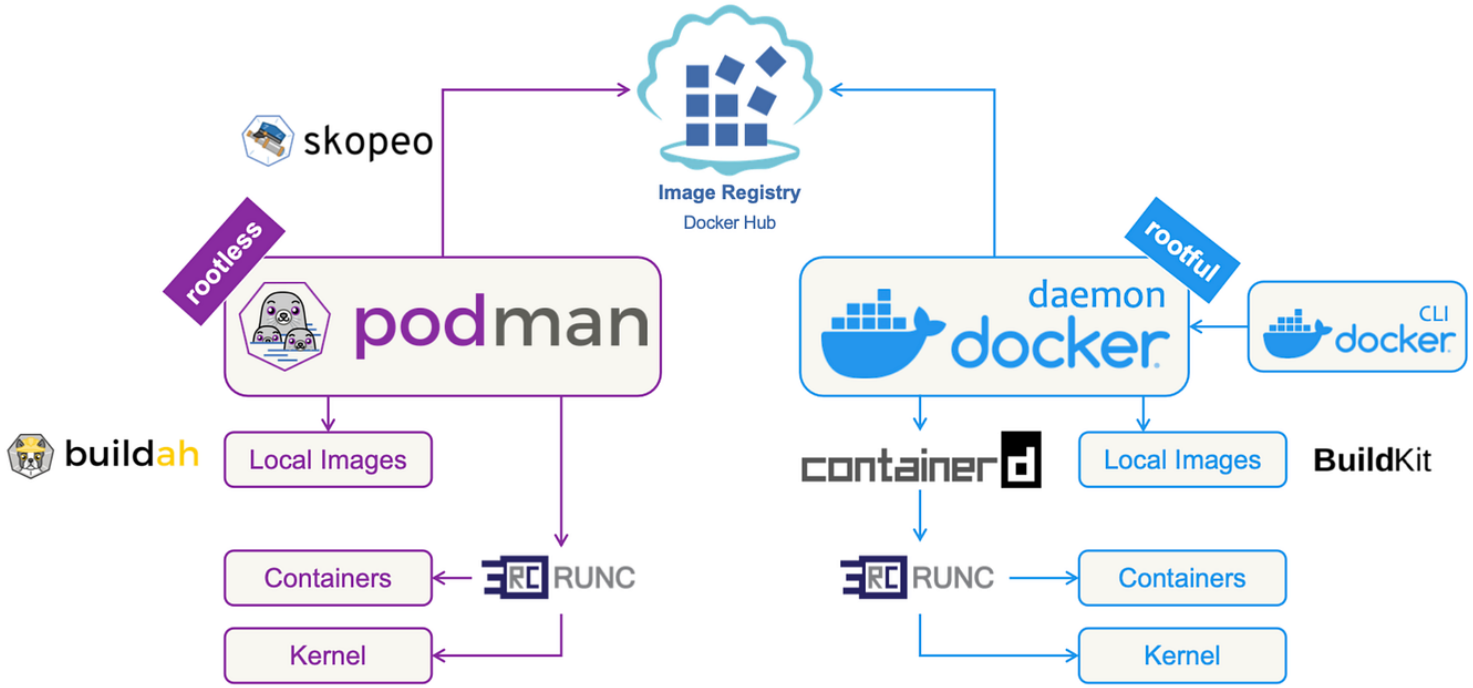
- Si un attaquant compromet le conteneur, il n'a pas les privilèges root sur l'hôte.

```
FROM python:3.11-slim

WORKDIR /app

# Créer un utilisateur non-root
RUN useradd --create-home appuser

COPY --chown=appuser:appuser requirements.txt .
RUN pip install -r requirements.txt

COPY --chown=appuser:appuser . .

# Utiliser cet utilisateur
USER appuser
```

Le container tourne en tant que appuser, pas root

Podman : alternative rootless

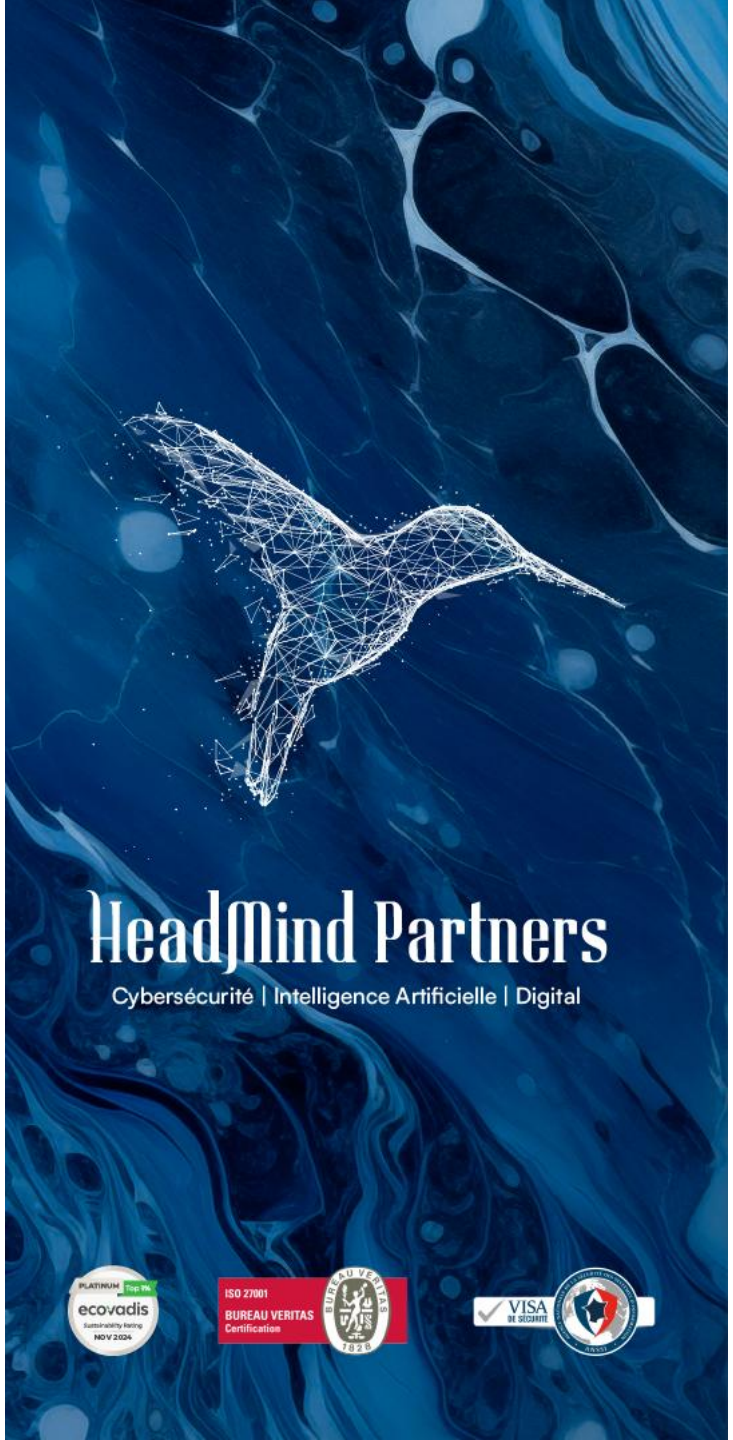
- Compatible Docker (mêmes commandes)
- Pas de daemon, pas de groupe privilégié
- Chaque user lance ses propres containers isolés
- Développé par Red Hat, utilisé en entreprise
- podman run = docker run



Podman : même syntaxe, plus sécurisé

```
# Installation
$ sudo apt install podman

# Même commandes que Docker
$ podman build -t mon-app .
$ podman run -p 8501:8501 mon-app

# Pas besoin de sudo ni de groupe docker
# Chaque utilisateur a son propre espace

# Alias pour compatibilité
$ alias docker=podman
```

Drop-in replacement pour Docker, sans les risques



10. Conclusion !

Ce qu'on a vu

- Docker résout "works on my machine"
- Images (recettes) vs Containers (instances)
- Dockerfile : FROM, COPY, RUN, CMD
- Layers et optimisation du cache
- Compose pour orchestrer plusieurs services
- Volumes pour persister les données
- Sécurité : le groupe docker est dangereux !

TD2 — 4 parties

- 1. Dockeriser ENSTArtup (Dockerfile, build, run)
- 2. Ajouter PostgreSQL avec Compose
- 3. CTF : exploiter le groupe docker via SSH
- 4. Remédiation avec Podman

Des questions ?